



南开大学  
Nankai University

南 开 大 学

计 算 机 学 院

深度学习实验报告

---

## 循环神经网络

---

2112213 冯思程

年级：2021 级

专业：计算机科学与技术

指导教师：侯淇彬

2025 年 5 月 29 日

## 摘要

本文围绕名字识别任务，探讨了 Vanilla RNN 与 LSTM 两种循环神经网络结构的应用效果，并进行了对比分析。实验中，本文基于 PyTorch 构建了基础 RNN 模型和两种 LSTM 模型（包括自定义实现和 PyTorch 内置实现），并评估它们在序列分类任务中的表现。结果表明，LSTM 借助其门控机制在处理长序列时具有更强的稳定性和表达能力，明显优于传统 RNN。同时，自行实现的 LSTM 模型在准确率和收敛性方面与官方实现基本一致，验证了其功能的正确性。此外，本文还简要分析了 LSTM 相比 RNN 的优势，并介绍了其常见变体之一——GRU，为后续研究提供参考方向。

**关键字：**RNN, LSTM, PyTorch, 名字识别, GRU

## 目录

|                                    |          |
|------------------------------------|----------|
| <b>一、 研究背景与准备</b>                  | <b>1</b> |
| (一) 研究背景 .....                     | 1        |
| (二) 数据集 .....                      | 1        |
| (三) 实验设置 .....                     | 1        |
| <b>二、 Vanilla RNN</b>              | <b>1</b> |
| (一) 网络结构 .....                     | 1        |
| (二) 实验结果 .....                     | 2        |
| <b>三、 LSTM</b>                     | <b>3</b> |
| (一) 网络结构 .....                     | 3        |
| (二) 实验结果 .....                     | 4        |
| <b>四、 讨论</b>                       | <b>6</b> |
| (一) 为什么 LSTM 网络的性能优于 RNN 网络? ..... | 6        |
| (二) GRU: 一种轻量级的 LSTM 变体 .....      | 6        |
| <b>五、 实验心得</b>                     | <b>6</b> |

## 一、 研究背景与准备

### (一) 研究背景

循环神经网络 (Recurrent Neural Networks, RNNs) 是处理序列数据的经典神经网络结构, 近年来在自然语言处理、语音识别和时间序列建模等领域取得了广泛应用。早期的研究 [3] 展示了 RNN 在处理时间序列任务中的潜力, 其通过引入循环连接, 使得模型具备记忆历史输入信息的能力, 从而适应变长序列的建模。然而, 标准 RNN 结构在面对长序列时容易出现梯度消失与梯度爆炸问题, 导致模型难以捕捉长期依赖关系。

为克服上述限制, Hochreiter 和 Schmidhuber 提出了长短期记忆网络 (Long Short-Term Memory, LSTM) [4]。该结构引入了输入门、遗忘门和输出门等门控机制, 能够有效选择性地保留或遗忘信息, 从而在较长序列中保持稳定的梯度传播。LSTM 被广泛用于语言建模、文本分类、语音识别等序列学习任务, 并被验证具有良好的性能表现。

此外, 为进一步简化网络结构并降低计算成本, Gated Recurrent Unit (GRU) 被提出 [2], 其保留了 LSTM 的主要优势, 同时减少了参数量 (细节请见 Section 二)。在此基础上, 诸如双向 RNN (Bidirectional RNN) [5] 和基于注意力机制的 RNN [1] 等变体也被广泛研究, 用以提升模型对上下文的理解能力。

本文以字符级文本分类为任务主线, 基于基础 RNN 构建字符序列分类模型, 并分析其在小规模数据集上的性能表现, 同时探讨更复杂结构 (如 LSTM) 在序列建模中的潜力, 为后续多模型对比实验奠定基础。

### (二) 数据集

本研究采用的实验数据集为人名分类数据集 (Names Classification Dataset)。该数据集由多个语言类别下的人名组成, 广泛用于字符级序列建模和文本分类任务, 特别适合用于测试循环神经网络 (RNN) 的序列处理能力。

### (三) 实验设置

本实验采用随机采样方式进行训练, 每轮迭代从训练集中随机选取一个样本进行更新, 累计训练迭代步数设为 300,000。训练过程中设置学习率为 0.005, 优化方式为手动梯度下降 (SGD), 损失函数为负对数似然损失 (NLLLoss)。为监控模型性能, 每间隔 1000 步记录一次平均训练损失与准确率, 并绘制成曲线进行分析。

模型训练与评估均在单张 RTX 4090 GPU 上完成。由于该实验使用的是字符级小样本分类任务, 未显式设置 batch size, 而是基于单样本逐步训练 (相当于 batch size 为 1)。为确保评估效果可靠, 实验在训练结束后通过 10,000 个随机样本构建混淆矩阵, 对模型的分类能力进行全面分析。

## 二、 Vanilla RNN

### (一) 网络结构

Vanilla RNN 架构较为简洁, 仅包含一个隐状态和输入的连接层。该模型接收每个字符的 one-hot 向量作为输入, 并结合上一个时间步的隐状态进行更新。具体网络结构如下:

RNN (

(i2h): Linear(in\_features=185, out\_features=128, bias=True)

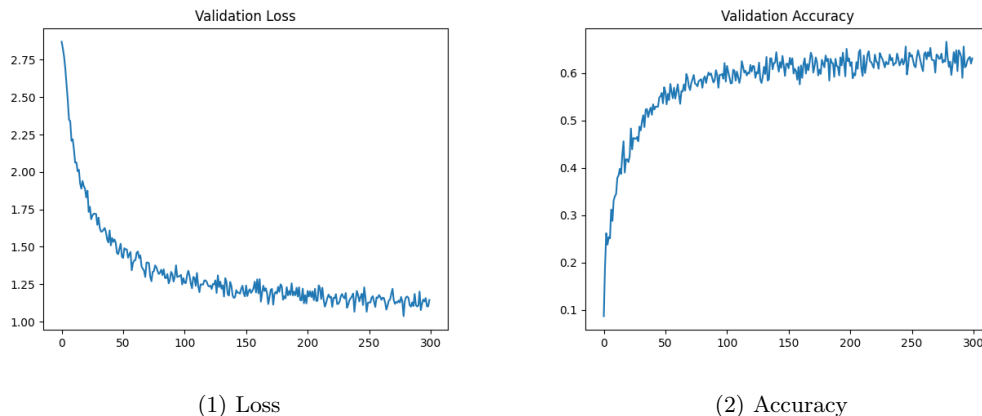


图 1: Vanilla RNN 训练过程的损失与准确率

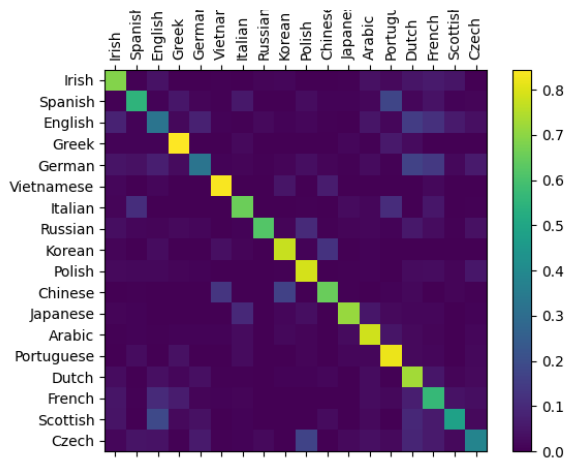


图 2: Vanilla RNN 混淆矩阵

```

(i2o): Linear(in_features=185, out_features=18, bias=True)
(softmax): LogSoftmax(dim=1)
)

```

其中，185 表示输入向量（字符 one-hot 向量）与上一时刻隐状态拼接后的维度，128 为隐藏层大小，18 为分类任务中的类别数（即语言类别）。模型通过 i2h 层更新隐藏状态，通过 i2o 层输出当前字符的分类结果，最后经过 log softmax 得到对各类别的概率分布。

## (二) 实验结果

如图 1 所示，模型训练过程中准确率逐步上升，最终在验证集上达到了约 63% 的准确率，损失值也从初始的接近 2.8 降低至约 1.1，说明模型具备较强的收敛能力。同时可以观察到训练曲线整体收敛平稳，未出现明显过拟合，模型在训练过程中稳定提升分类能力。

从图 2 的混淆矩阵可以看出，模型在大多数语言类别上的分类表现较好。特别是希腊语 (Greek) 和波兰语 (Polish) 的对角线块较亮，表示模型在这些类别上的准确率较高。同时也可以观察到某些类别之间存在明显混淆，例如英语 (English) 与德语 (German) 的对角块相对较暗，表明模型在这两个语言上的分类性能稍弱，可能是由于这两种语言的人名拼写存在部分相似性。此外，部分语种如法语 (French) 与葡萄牙语 (Portuguese) 等，也存在一定程度的分类混

淆，这与其语言结构相近有关，模型在处理这类边界模糊的语言时识别难度更高。

综上所述，Vanilla RNN 在字符级语言分类任务中展现出一定的建模能力，尤其在部分语言上具有较高的识别准确率，但在处理语言间相似性较高的类别时仍存在一定误判。

### 三、 LSTM

#### (一) 网络结构

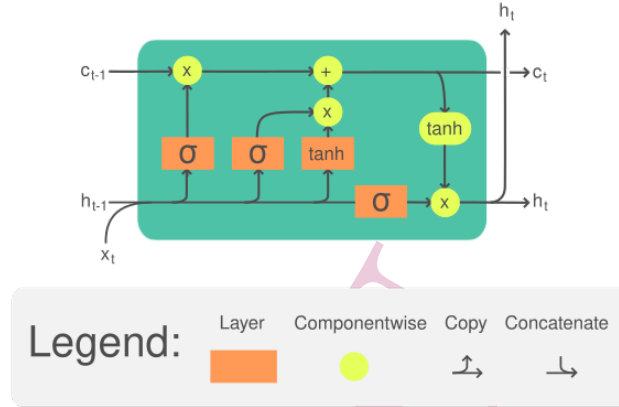


图 3: LSTM 的架构图

LSTM (Long Short-Term Memory) 是一种经典的循环神经网络结构 (图 3)，专为解决传统 RNN 在长序列建模中存在的梯度消失问题而设计。其核心思想是通过引入**门控机制**控制信息的流动，从而实现对重要信息的长期记忆与不重要信息的遗忘。LSTM 包含三个主要的门控结构：

- **遗忘门 (Forget Gate)**：决定当前时间步应从上一时刻的记忆状态中遗忘多少信息：

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

- **输入门 (Input Gate)**：决定当前输入中哪些信息被写入记忆状态：

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

- **输出门 (Output Gate)**：控制当前隐藏状态的输出内容：

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

综合上述门控操作，LSTM 在每个时间步更新记忆状态  $c_t$  和隐藏状态  $h_t$  的方式如下：

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$

其中， $\sigma$  表示 Sigmoid 激活函数， $\odot$  表示逐元素乘法操作， $x_t$  是当前输入， $h_{t-1}$  是上一时刻的隐藏状态， $c_{t-1}$  是上一时刻的记忆状态。LSTM 能够捕捉较长时间跨度内的依赖关系，广泛应用于语言建模、文本分类、时间序列预测等序列任务。

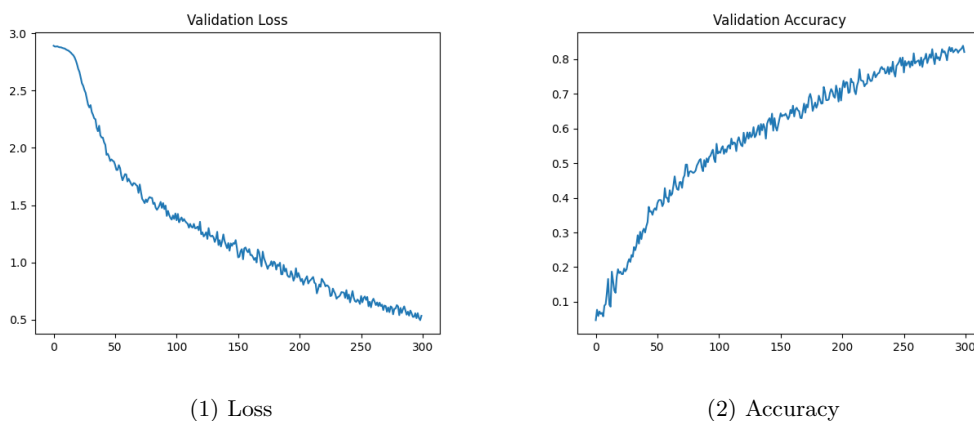


图 4: LSTM 在训练过程的损失与准确率 (PyTorch)

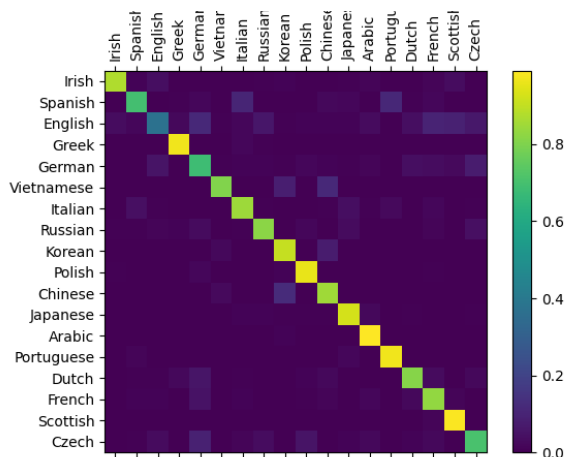


图 5: LSTM 混淆矩阵 (PyTorch)

## (二) 实验结果

这里我对比了通过 PyTorch 实现 LSTM 和自行实现了 LSTM 的实验结果，下面是各自的结果与分析。

**PyTorch 版本性能分析** 从图 4 中可以看出，模型在训练过程中收敛良好，最终一个 Epoch 的准确率达到 82%，相较于原始的 Vanilla RNN 有显著提升。

具体而言，LSTM 模型的损失值在训练后期明显低于 RNN，约为其一半左右，说明模型更有效地学习到了序列中的特征信息。准确率曲线也显示出整体稳定上升，最终取得较高准确率，进一步验证了 LSTM 的优势。

从混淆矩阵（图 5）可以看出，大多数语言类别在对角线上的色块较为明亮，表示模型在分类上的准确性和稳定性较强。整体来看，LSTM 在处理字符级名字识别任务中展现出更优的性能，尤其在捕捉长序列信息和提升分类精度方面明显优于原始的 RNN 结构。

**自己实现版本性能分析** 通过运行自行实现的 LSTM 模型，得到了图 6 和图 7 的结果。整体曲线变化和混淆矩阵的变化趋势和数值结果都是相似的。

从损失曲线可以看出，训练过程中 Loss 降低较快且数值较小，说明模型具有较好的学习能力和收敛效果。同时，准确率曲线整体走势平稳，波动较小，反映出模型训练过程较为稳定，具

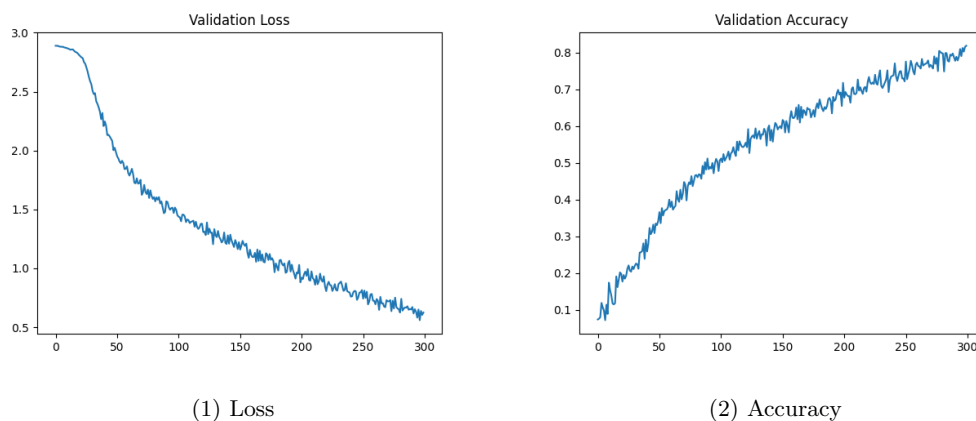


图 6: LSTM 在训练过程的损失与准确率 (自己实现)

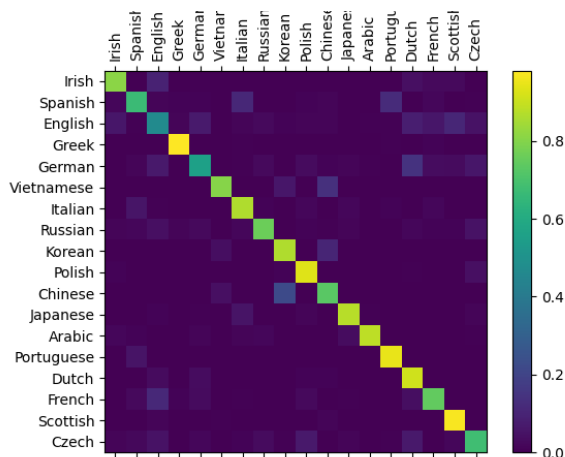


图 7: LSTM 混淆矩阵 (自己实现)

备良好的泛化能力。综合来看，自实现的 LSTM 模型在性能上达到了预期目标，能够有效完成字符级语言分类任务。

**性能对比与结果总结** 在对各模型进行测试后，我们对其性能表现与训练耗时进行了整理。从整体结果来看，LSTM 在准确率和收敛性方面均明显优于原始的 Vanilla RNN，虽然训练时间略有增加，但差异并不明显，计算开销在可接受范围内，同时这里我自行实现的 LSTM 要比调用 PyTorch 的 LSTM 略慢，这与 PyTorch 有一些加速有关。

此外，自行实现的 LSTM 与使用 PyTorch 内置 `nn.LSTM` 的模型在准确率和训练时间上表现非常接近，进一步验证了自主实现的正确性。两者之间轻微的性能差距可能来源于标准库实现中采用了更优的初始化策略（如 Kaiming 初始化）以及其他工程技巧，例如梯度裁剪或更稳定的激活函数设计，这些通常有助于提升模型的数值稳定性与训练效率。

## 四、 讨论

### (一) 为什么 LSTM 网络的性能优于 RNN 网络?

LSTM 网络在序列建模任务中表现优于传统 RNN，其核心原因在于结构上的改进使其更适合捕捉长期依赖关系。在本研究的人名语言分类任务中，LSTM 在大多数语言类别上取得更高的准确率，尤其是在英语和德语等语言相近的类别上，相比 RNN 具备更强的区分能力。

具体而言，LSTM 网络在以下几个方面具备显著优势：

- **长期依赖建模能力强**：传统 RNN 难以有效捕捉长序列中的远距离依赖，尤其在序列较长或特征相似的语言（如英语与德语）上表现不佳。LSTM 通过引入细胞状态（Cell State）作为信息通道，能够在多个时间步中保留关键上下文信息，显著提升模型对长期依赖的建模能力。
- **门控机制增强信息控制**：LSTM 引入遗忘门、输入门和输出门，允许模型在每一步对信息的保留、更新和输出进行精细控制。这种机制相比 RNN 的简单状态传播方式更具灵活性，能有效筛选对分类有用的关键信息。
- **缓解梯度消失问题**：由于结构设计上的改进，LSTM 能在反向传播过程中保持较为稳定的梯度流动，避免传统 RNN 在长序列训练中面临的梯度消失问题，提高了模型的训练效率和收敛能力。
- **增强模型泛化能力**：通过记忆单元与门控的结合，LSTM 能在复杂数据分布下保留有用特征，同时抑制噪声信息，从而在多个语言类别上均取得更稳健的分类效果。

实验结果表明，LSTM 在整体准确率、语言间可分性以及长序列处理能力上更具优势，是更适合序列分类任务的结构。但是在后期，Transformer 的出现逐渐占据了优势地位，其可以更好地建模序列问题。

### (二) GRU：一种轻量级的 LSTM 变体

在 LSTM 被提出后，研究者们针对其结构复杂、计算开销较大的问题，提出了多种改进方案，其中较具代表性的就是门控循环单元（Gated Recurrent Unit, GRU）[2]。GRU 是一种简化版的 RNN 结构，由 Cho 等人于 2014 年提出，旨在保留 LSTM 优势的同时减少结构复杂度。

与 LSTM 不同，GRU 仅使用两个门控机制：更新门（update gate）和重置门（reset gate）。更新门用于控制当前时间步是否保留前一时刻的隐藏状态，而重置门则决定在生成候选状态时需忽略多少历史信息。

具体来说，GRU 首先根据当前输入与前一隐藏状态计算两个门的值；接着通过重置门调节前一隐藏状态的影响，并与当前输入一同生成候选隐藏状态；最后，利用更新门在旧隐藏状态和新候选状态之间进行加权更新，生成新的隐藏状态。

由于结构更为紧凑，GRU 的参数数量少于 LSTM，训练速度更快，尤其适合对资源要求较高或数据量有限的场景。在许多序列建模任务中，GRU 也能达到与 LSTM 相当的性能表现，因此被广泛应用于实际系统中。

## 五、 实验心得

通过本次实验，我深入理解了 Vanilla RNN 的基本原理，并掌握了如何使用 PyTorch 构建循环神经网络来完成名字识别任务。在此基础上，我尝试自行实现了 LSTM 结构，而非直接使

用教师提供的基于 `nn.LSTM` 的封装代码，进一步加深了对 LSTM 内部机制的理解。此外，我还对一些改进方法（GRU）进行了延伸思考，收获颇丰，为我今后在深度学习方向的学习与研究打下了良好基础。

NIKU

## 参考文献

- [1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.
- [2] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *EMNLP*, 2014.
- [3] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.
- [4] Alex Graves and Alex Graves. Long short-term memory. *Supervised sequence labelling with recurrent neural networks*, pages 37–45, 2012.
- [5] Mike Schuster and Kuldip K Paliwal. Bidirectional recurrent neural networks. *IEEE transactions on Signal Processing*, 45(11):2673–2681, 1997.

MINIB